

Candidate Submission

SYSTEM DESIGN

API System Design & Endpoint Specification

System architecture, data model and full endpoint specification for the ProjectFlow API — covering authentication, authorization, task assignment, and the task-activity feed.

Prepared by

Youssef Bassiouny

Project

ProjectFlow — Full-Stack Engineer Assessment

Date

September 2026

CONTENTS

ProjectFlow API System Design & Endpoint Specification

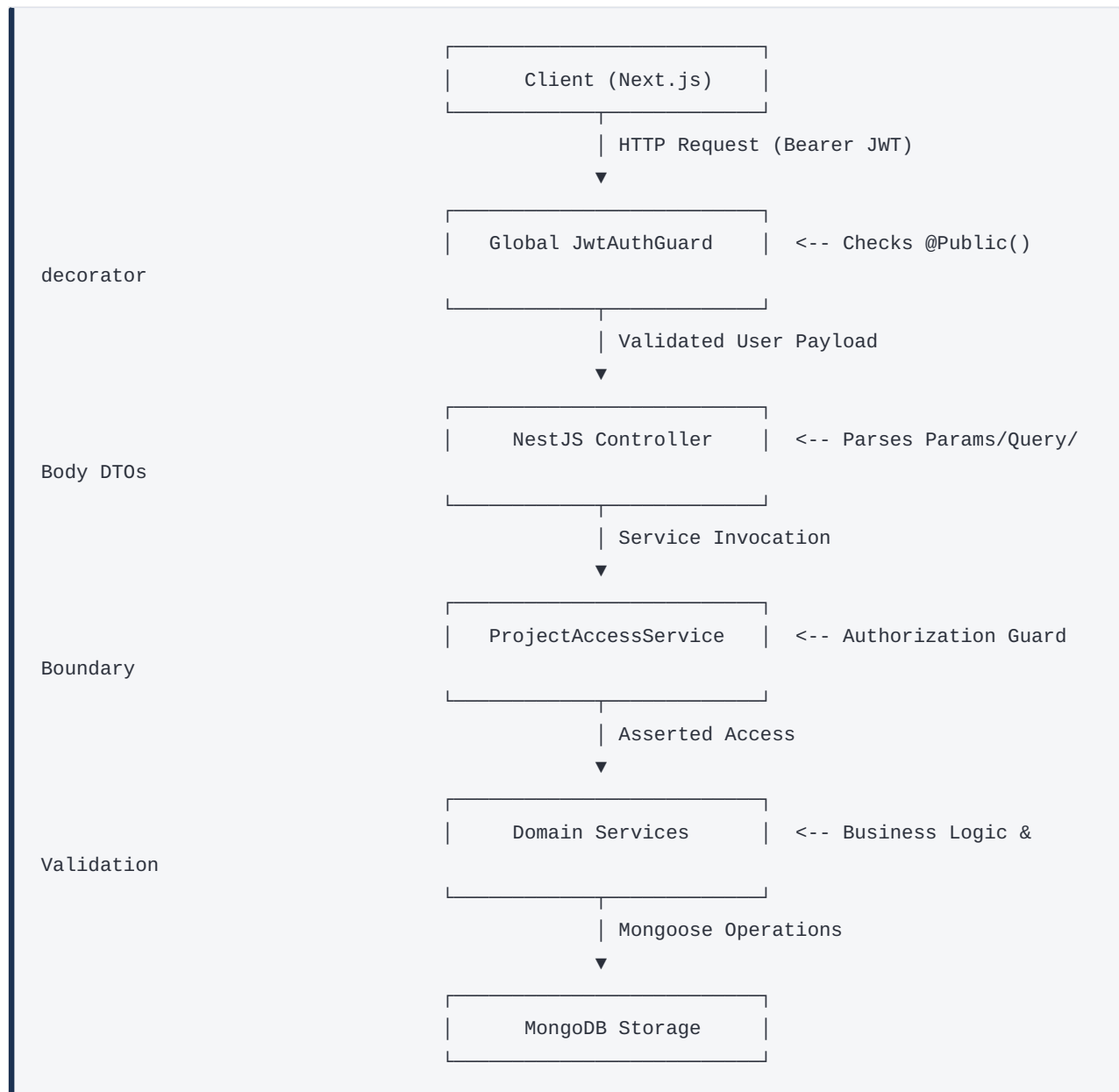
1. System Overview & Architecture
2. Authentication & Authorization Architecture
3. Data Schemas & Database Entities
4. API Endpoint Specification (Requests & Responses)
5. Concurrency & Data Consistency Mechanics

ProjectFlow API System Design & Endpoint Specification

1. System Overview & Architecture

ProjectFlow is a TypeScript monorepo built using NestJS for the backend API, Next.js (App Router) for the web application, and MongoDB (via Mongoose) for persistence. Shared types, DTOs, and utility interfaces are maintained within [@projectflow/shared](#).

High-Level Request Flow Architecture



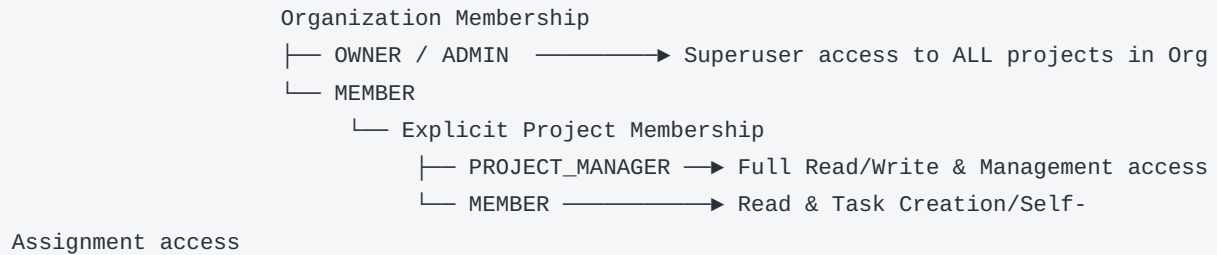
2. Authentication & Authorization Architecture

2.1 Authentication

- **Mechanism**: JSON Web Tokens (JWT) passed via the `Authorization: Bearer <token>` header.
- **Global Guard**: `JwtAuthGuard` guards all routes by default unless explicitly decorated with `@Public()`.
- **User Context**: Handled via the custom `@CurrentUser()` decorator, which extracts the authenticated user ID from the request state.

2.2 Authorization Model

Authorization is evaluated at the service boundary using `ProjectAccessService` :



AUTHORIZATION RULES TABLE

Role / Context	View Project & Tasks	Create Tasks	Modify Task Details	Update Task Status	Assign Other Members	Self Assign Task	Manage Project Members
Org OWNER / ADMIN	Yes	Yes	Yes	Yes	Yes	Yes	Yes
Project MANAGER	Yes	Yes	Yes	Yes	Yes	Yes	Yes
Project MEMBER	Yes	Yes	Creator Only	Yes (Authorized)	No	Yes	No
Non-Project Member	No	No	No	No	No	No	No

3. Data Schemas & Database Entities

3.1 Schemas Overview

USER

```
{
  _id: ObjectId,
  name: string,
  email: string (unique, lowercase),
  passwordHash: string,
  avatarUrl: string | null,
  createdAt: Date,
  updatedAt: Date
}
```

ORGANIZATION

```
{
  _id: ObjectId,
  name: string,
  slug: string (unique, lowercase),
  createdBy: ObjectId (ref: 'User'),
  createdAt: Date,
  updatedAt: Date
}
```

ORGANIZATIONMEMBER

```
{
  _id: ObjectId,
  organizationId: ObjectId (ref: 'Organization'),
  userId: ObjectId (ref: 'User'),
  role: 'OWNER' | 'ADMIN' | 'MEMBER',
  createdAt: Date,
  updatedAt: Date
}
// Indexes: { organizationId: 1, userId: 1 } (unique)
```

PROJECT

```
{
  _id: ObjectId,
  organizationId: ObjectId (ref: 'Organization'),
  name: string,
  key: string (uppercase),
  description: string | null,
  createdBy: ObjectId (ref: 'User'),
  createdAt: Date,
  updatedAt: Date
}
// Indexes: { organizationId: 1, key: 1 } (unique)
```

PROJECTMEMBER

```
{
  _id: ObjectId,
  projectId: ObjectId (ref: 'Project'),
  userId: ObjectId (ref: 'User'),
  role: 'PROJECT_MANAGER' | 'MEMBER',
  createdAt: Date,
  updatedAt: Date
}
// Indexes: { projectId: 1, userId: 1 } (unique)
```

TASK

```
{
  _id: ObjectId,
  projectId: ObjectId (ref: 'Project'),
  number: number,
  key: string,
  title: string,
  description: string | null,
  status: 'TODO' | 'IN_PROGRESS' | 'IN_REVIEW' | 'DONE',
  priority: 'LOW' | 'MEDIUM' | 'HIGH' | 'URGENT',
  createdBy: ObjectId (ref: 'User'),
  assignee: ObjectId | null (ref: 'User'),
  createdAt: Date,
  updatedAt: Date
}
// Indexes:
// - { projectId: 1, number: 1 } (unique - concurrency protection)
// - { projectId: 1, status: 1 }
// - { createdAt: -1 }
```

TASKSEQUENCE (ATOMIC CONCURRENCY COUNTER)

```
{
  _id: ObjectId,
  projectId: ObjectId (ref: 'Project', unique),
  seq: number
}
```

TASKACTIVITY (ASSIGNEE ACTIVITY LOG)

```
{
  _id: ObjectId,
  taskId: ObjectId (ref: 'Task'),
  projectId: ObjectId (ref: 'Project'),
  actorId: ObjectId (ref: 'User'),
  type: 'TASK_ASSIGNEE_CHANGED',
  metadata: {
    from: ObjectId | null,
    to: ObjectId | null
  },
  createdAt: Date
}
// Indexes: { taskId: 1, createdAt: -1 }
```

COMMENT

```
{
  _id: ObjectId,
  taskId: ObjectId (ref: 'Task'),
  authorId: ObjectId (ref: 'User'),
  content: string,
  createdAt: Date,
  updatedAt: Date
}
// Indexes: { taskId: 1, createdAt: 1 }
```

4. API Endpoint Specification (Requests & Responses)

Common Standard Formats

STANDARD ERROR RESPONSE (APIERRORBODY)

```
{
  "statusCode": 403,
  "message": "You do not have access to this project",
  "error": "Forbidden"
}
```

STANDARD PAGINATED RESPONSE (`PAGINATED<T>`)

```
{
  "items": [],
  "total": 0,
  "page": 1,
  "pageSize": 20
}
```

4.1 Authentication Endpoints (`/auth`)**POST `/AUTH/REGISTER`**

- Auth: Public (`@Public()`)
- Request Body:

```
{
  "name": "Jane Doe",
  "email": "jane@example.com",
  "password": "securepassword123"
}
```

- Response Body (`201 Created` - `AuthSession`):

```
{
  "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6I..",
  "user": {
    "id": "660000000000000000000001",
    "name": "Jane Doe",
    "email": "jane@example.com",
    "avatarUrl": null
  }
}
```

POST `/AUTH/LOGIN`

- Auth: Public (`@Public()`)
- Request Body:

```
{
  "email": "jane@example.com",
  "password": "securepassword123"
}
```

- Response Body (`200 OK` - `AuthSession`):

```
{
  "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6I..",
  "user": {
    "id": "66000000000000000000000000000001",
    "name": "Jane Doe",
    "email": "jane@example.com",
    "avatarUrl": null
  }
}
```

GET /AUTH/ME

- Auth: Bearer JWT
- Response Body (200 OK - `CurrentUser`):

```
{
  "id": "66000000000000000000000000000001",
  "name": "Jane Doe",
  "email": "jane@example.com",
  "avatarUrl": null,
  "organizations": [
    {
      "id": "66000000000000000000000000000010",
      "name": "Acme Corp",
      "slug": "acme-corp",
      "role": "OWNER"
    }
  ]
}
```

4.2 Organizations Endpoints (/organizations)

GET /ORGANIZATIONS

- Auth: Bearer JWT
- Response Body (200 OK - `Array<OrganizationSummary & { role: OrganizationRole }>`):

```
[
  {
    "id": "66000000000000000000000000000010",
    "name": "Acme Corp",
    "slug": "acme-corp",
    "role": "OWNER"
  }
]
```


[illegible]

GET /PROJECTS/:PROJECTID

- **Auth:** Bearer JWT (Project Member / Org Elevated)
- **Response Body** (200 OK - ProjectDetail)

GET /PROJECTS/:PROJECTID/MEMBERS

- **Auth:** Bearer JWT (Project Member / Org Elevated)
- **Response Body** (`200 OK` - `ProjectMemberEntry[]`):

```
[
  {
    "id": "6600000000000000000000000000200",
    "projectId": "6600000000000000000000000000100",
    "role": "PROJECT_MANAGER",
    "user": {
      "id": "6600000000000000000000000000001",
      "name": "Jane Doe",
      "email": "jane@example.com",
      "avatarUrl": null
    },
    "createdAt": "2026-09-15T09:00:00.000Z"
  }
]
```

POST /PROJECTS/:PROJECTID/MEMBERS

- **Auth:** Bearer JWT (Project Manager / Org Elevated)
- **Request Body:**

```
{  
  "userId": "6600000000000000000000000002",  
  "role": "MEMBER"  
}
```

- **Response Body** (201 Created - ProjectMemberEntry)
-

4.4 Tasks Endpoints (/tasks)

GET /PROJECTS/:PROJECTID/TASKS

- **Auth:** Bearer JWT (Project Member / Org Elevated)
- **Query Parameters:**
 - **page** (optional, default: 1)
 - **pageSize** (optional, default: 20)
 - **status** (optional: TODO | IN_PROGRESS | IN_REVIEW | DONE)
 - **priority** (optional: LOW | MEDIUM | HIGH | URGENT)
- **Response Body** (200 OK - Paginated<TaskSummary>):

```
{  
  "items": [  
    {  
      "id": "6600000000000000000000000000300",  
      "projectId": "6600000000000000000000000000100",  
      "number": 1,  
      "key": "ENG-1",  
      "title": "Setup CI/CD Pipeline",  
      "status": "IN_PROGRESS",  
      "priority": "HIGH",  
      "commentCount": 3,  
      "createdBy": {  
        "id": "6600000000000000000000000000001",  
        "name": "Jane Doe",  
        "email": "jane@example.com",  
        "avatarUrl": null  
      },  
      "assignee": {  
        "id": "6600000000000000000000000000002",  
        "name": "John Smith",  
        "email": "john@example.com",  
        "avatarUrl": null  
      },  
      "createdAt": "2026-09-15T10:00:00.000Z",  
      "updatedAt": "2026-09-15T11:00:00.000Z"  
    }  
  ],  
  "total": 1,  
  "page": 1,  
  "pageSize": 20  
}
```

POST /PROJECTS/:PROJECTID/TASKS

- **Auth:** Bearer JWT (Project Member / Org Elevated)
- **Request Body:**

```
{
  "title": "Setup CI/CD Pipeline",
  "description": "Configure GitHub Actions workflows",
  "priority": "HIGH",
  "status": "TODO",
  "assigneeId": "660000000000000000000002"
}
```

- **Response Body** (201 Created - TaskDetail):

```
{
  "id": "66000000000000000000000000300",
  "projectId": "66000000000000000000000000100",
  "number": 1,
  "key": "ENG-1",
  "title": "Setup CI/CD Pipeline",
  "description": "Configure GitHub Actions workflows",
  "status": "TODO",
  "priority": "HIGH",
  "commentCount": 0,
  "createdBy": {
    "id": "66000000000000000000000000001",
    "name": "Jane Doe",
    "email": "jane@example.com"
  },
  "assignee": {
    "id": "660000000000000000000000000002",
    "name": "John Smith",
    "email": "john@example.com"
  },
  "project": {
    "id": "6600000000000000000000000000100",
    "name": "Internal Platform",
    "key": "ENG"
  },
  "createdAt": "2026-09-15T10:00:00.000Z",
  "updatedAt": "2026-09-15T10:00:00.000Z"
}
```

GET /TASKS/:TASKID

- **Auth:** Bearer JWT (Project Member / Org Elevated)
- **Response Body** (200 OK - TaskDetail)

PATCH /TASKS/:TASKID

- **Auth:** Bearer JWT (Task Creator / Project Manager / Org Elevated)
- **Request Body:**

```
{
  "title": "Updated Task Title",
  "description": "Updated Task Description",
  "priority": "URGENT"
}
```

- **Response Body** (200 OK - TaskDetail)

PATCH /TASKS/:TASKID/STATUS

- **Auth:** Bearer JWT (Enforces Project Access Check via **ProjectAccessService**)
- **Request Body:**

```
{  
  "status": "DONE"  
}
```

- **Response Body** (200 OK - TaskDetail)

PATCH /TASKS/:TASKID/ASSIGNEE

- **Auth:** Bearer JWT (Authorized Role or Self-Assignment by Member)
- **Request Body:**

```
{  
  "assigneeId": "660000000000000000000002"  
}
```

(Pass **assigneeId: null** to unassign)

- * **Response Body** (200 OK - TaskDetail)

GET /TASKS/:TASKID/ACTIVITY

- **Auth:** Bearer JWT (Project Member / Org Elevated)
- **Query Parameters:** **page** , **pageSize**
- **Response Body** (200 OK - Paginated<TaskActivityEntry>):

```
{
  "items": [
    {
      "id": "6600000000000000000000000000400",
      "taskId": "6600000000000000000000000000300",
      "actor": {
        "id": "6600000000000000000000000000001",
        "name": "Jane Doe",
        "email": "jane@example.com"
      },
      "type": "TASK_ASSIGNEE_CHANGED",
      "metadata": {
        "from": null,
        "to": {
          "id": "6600000000000000000000000000002",
          "name": "John Smith",
          "email": "john@example.com"
        }
      },
      "createdAt": "2026-09-15T11:00:00.000Z"
    }
  ],
  "total": 1,
  "page": 1,
  "pageSize": 20
}
```

DELETE /TASKS/:TASKID

- **Auth:** Bearer JWT (Project Manager / Org Elevated)
- **Response:** 204 No Content

4.5 Comments Endpoints (/tasks/:taskId/comments)

GET /TASKS/:TASKID/COMMENTS

- **Auth:** Bearer JWT (Project Member / Org Elevated)
- **Query Parameters:** page , pageSize
- **Response Body** (200 OK - Paginated<CommentEntry>):

```
{
  "items": [
    {
      "id": "660000000000000000000000500",
      "taskId": "660000000000000000000000300",
      "content": "Looking into this setup today.",
      "author": {
        "id": "660000000000000000000000002",
        "name": "John Smith",
        "email": "john@example.com"
      },
      "createdAt": "2026-09-15T11:30:00.000Z",
      "updatedAt": "2026-09-15T11:30:00.000Z"
    }
  ],
  "total": 1,
  "page": 1,
  "pageSize": 20
}
```

POST /TASKS/:TASKID/COMMENTS

- Auth: Bearer JWT (Project Member / Org Elevated)
- Request Body:

```
{
  "content": "Looking into this setup today."
}
```

- Response Body (201 Created - CommentEntry)

5. Concurrency & Data Consistency Mechanics

Atomic Task Key Generation

To resolve the race condition present in `countDocuments({ projectId }) + 1`, sequence allocation is handled via MongoDB atomic operations:

```
const sequence = await this.taskSequenceModel.findOneAndUpdate(
  { projectId },
  { $inc: { seq: 1 } },
  { new: true, upsert: true }
);
const number = sequence.seq;
const key = `${project.key}-${number}`;
```

Combined with the MongoDB compound unique index:

```
TaskSchema.index({ projectId: 1, number: 1 }, { unique: true });
```

This guarantees strict sequential numbering without duplicate key generation under high concurrency.